

BUHLOWAR

v2.0 - ESP32-S3

Полный анализ исходного кода

Системы автоматизации ректификации и дистилляции

Репозиторий: github.com/Vlad-alco/BuhloWar

Платформа: ESP32-S3 (dual-core, FreeRTOS)

Дата анализа: 09.07.2026

Показатель	Значение
Общий объем кода	~490 КБ (35 файлов)
Основной язык	C++ (Arduino/ESP32)
Критических проблем	8
Проблем среднего уровня	14
Рекомендаций по улучшению	12

Содержание

1. Обзор архитектуры системы

2. Подробный разбор модулей

- 2.1. BuhloWar.ino - Точка входа и главный цикл
- 2.2. ProcessEngine - Ядро обработки процессов
- 2.3. AppNetwork - Сетевой менеджер и Web-интерфейс
- 2.4. OutputManager - Управление выходами
- 2.5. SensorManager - Управление датчиками
- 2.6. SDLogger - Логирование на SD-карту
- 2.7. CloudManager - Облачное подключение
- 2.8. ConfigManager - Хранение настроек (EEPROM)

3. Анализ процессов дистилляции и ректификации

- 3.1. Дистилляция: этапы и логика
- 3.2. Ректификация: этапы и логика
- 3.3. Система безопасности (Safety)

4. Выявленные баги и уязвимости

- 4.1. Утечки памяти и фрагментация кучи
- 4.2. Риск зависания системы
- 4.3. Проблемы многозадачности и синхронизации
- 4.4. Уязвимости безопасности
- 4.5. Логические ошибки управления

5. Сводная таблица проблем

6. Рекомендации по улучшению

1. Обзор архитектуры системы

BuhloWar - это система автоматического управления процессами дистилляции и ректификации спиртных напитков, работающая на микроконтроллере ESP32-S3. Система управляет нагревательными элементами (ТЭН), электромагнитными клапанами, датчиками температуры (DS18B20), датчиком давления/влажности (BME280), дисплеем LCD 2004 и веб-интерфейсом. Архитектура спроектирована с использованием двухядерной структуры FreeRTOS, где Core 0 отвечает за сетевое взаимодействие, а Core 1 обрабатывает основную логику управления в loop().

Аппаратная платформа

ESP32-S3 выбран как основа благодаря наличию двух ядер, встроенного Wi-Fi/Bluetooth, достаточного объема SRAM (512 КБ) и поддержки FreeRTOS. Контроллер управляет следующими периферийными устройствами: 4 цифровыми датчиками температуры DS18B20 по шине 1-Wire (TSA - температура царги, AQUA - температура дистиллята, TSAR - температура царги для ректификации, TANK - температура в кубе), барометрическим датчиком BME280 (давление, температура корпуса, влажность) по I2C, RTC часами DS3231, дисплеем LCD 2004 по I2C, твердотельными реле (SSR) для управления нагревателями и клапанами 220В, зуммером, и картой MicroSD для логирования через шину SPI.

Программная архитектура

Программная архитектура построена по модульному принципу с четким разделением ответственностей между компонентами. Главный файл BuhloWar.ino содержит функцию setup() для инициализации всех подсистем и loop() для основного цикла обработки. Обмен данными между ядрами FreeRTOS реализован через очередь команд (FreeRTOS Queue), куда сетевая задача на Core 0 помещает команды, полученные через веб-интерфейс или облачный менеджер, а основной цикл на Core 1 извлекает и обрабатывает их. Для защиты SD-карты от одновременного доступа с разных ядер используется мьютекс sdMutex, создаваемый в setup() до любой инициализации SD.

Архитектура включает следующие ключевые модули: ProcessEngine - ядро обработки процессов (разгон, отбор, стабилизация, головы, тело); OutputManager - управление физическими выходами (реле, клапаны, нагрев); SensorManager - опрос и фильтрация данных с датчиков температуры; SensorAdapter - адаптация данных от SensorManager в унифицированную структуру; AppNetwork - веб-сервер, WiFi-подключение и DNS-сервер для captive portal; CloudManager - асинхронная отправка телеметрии в облако через отдельную FreeRTOS задачу; SDLogger - ротлируемое логирование на SD-карту; ConfigManager - хранение настроек в EEPROM; набор меню для LCD-дисплея (MainMenu, DistMenu, RectMenu, SettingsMenu, SensorsMenu, ValveCalMenu, SetPwAsMenu).

Компонент	Интерфейс	Пин GPIO	Назначение
DS18B20 x4	1-Wire	GPIO3	Датчики температуры
BME280	I2C (0x76)	GPIO8/9	Давление, влажность
DS3231	I2C (0x68)	GPIO8/9	Часы реального времени

Компонент	Интерфейс	Пин GPIO	Назначение
LCD 2004	I2C (0x27)	GPIO8/9	Дисплей 20x4 символа
MicroSD	SPI (HSPI)	GPIO12-15	Логирование
SSR нагрев 1	Digital	GPIO46	ТЭН рабочий канал
SSR нагрев 2	Digital	GPIO18	ТЭН разгонный канал
Контактор	Digital	GPIO42	Главный силовой 220В
Клапан голов	Digital	GPIO37	Управление отбором голов
Клапан тела	Digital	GPIO38	Управление отбором тела
Клапан воды	Digital	GPIO2	Подача охлаждающей воды
Мешалка	Digital	GPIO47	Перемешивание в кубе
Зуммер	Digital	GPIO21	Звуковая сигнализация

Таблица 1. Аппаратные компоненты и их подключение

2. Подробный разбор модулей

2.1. BuhloWar.ino - Точка входа и главный цикл

Файл BuhloWar.ino является точкой входа приложения Arduino. Функция `setup()` выполняет строго упорядоченную инициализацию всех подсистем. Сначала создается мьютекс `sdMutex` для защиты SD-карты, затем инициализируется очередь команд `commandQueue` (32 элемента, `sizeof(CommandMessage)`), после чего вызывается `initSD()` для раннего запуска SD-карты и `startWebServerEarly()` для немедленного запуска веб-сервера в AP-режиме. Это позволяет пользователю получить доступ к веб-интерфейсу по адресу 192.168.4.1 еще до попытки подключения к WiFi, что значительно ускоряет первичную настройку устройства.

Далее `setup()` анализирует причину последнего сброса через `esp_reset_reason()`, синхронизирует время с DS3231, инициализирует I2C, настраивает кнопки с `INPUT_PULLUP`, загружает конфигурацию из EEPROM, инициализирует датчики, запускает сетевую задачу на Core 0 и подключается к WiFi. Важной особенностью является проверка призрачного процесса: если при включении обнаруживается флаг запущенного процесса (например, из-за внезапного отключения питания), система автоматически сбрасывает его для предотвращения неконтролируемого поведения оборудования.

Функция `loop()` работает на Core 1 и выполняет следующие шаги каждый цикл: (1) обработка команд из очереди через `xQueueReceive` с нулевым таймаутом (неблокирующе), (2) вызов `processEngine.update()` для обновления логики управления, (3) синхронизация с облаком через `cloudManager.update()`, (4) обновление выходов через `outputManager.update()`, (5) синхронизация состояния сети для отображения на LCD, (6) управление переключением экранов процессов, (7) обновление датчиков через `sensorAdapter.update()`, (8) проверка кнопок, (9) обновление LCD-дисплея с интервалом 500 мс. Диагностика времени выполнения цикла логируется при превышении 10 мс, что помогает выявлять медленные итерации.

2.2. ProcessEngine - Ядро обработки процессов

ProcessEngine (ProcessEngine.h/cpp, ~1966 строк) является центральным модулем системы, реализующим конечный автомат для управления процессами дистилляции и ректификации. Класс содержит 16 состояний (Stage): IDLE, WATER_TEST, RAZGON, WAITING, OTBOR, REPLACEMENT, BAKSTOP, NASEBYA, VALVE_CAL, SET_PW_AS, GOLOVY, GOLOVY_OK, TELO и FINISHING_WORK. Каждое состояние имеет свой обработчик, вызываемый из метода update() через switch-case.

Модуль поддерживает два типа процессов: дистилляцию (PROCESS_DIST) и ректификацию (PROCESS_RECT). Дистилляция проходит через этапы: WATER_TEST (проверка наличия воды) -> RAZGON (нагрев до заданной температуры) -> WAITING (ожидание прогрева царги на 5 градусов от начальной) -> OTBOR (отбор дистиллята) -> REPLACEMENT (сменная тары при достижении midterm-температуры с коррекцией по давлению) -> BAKSTOP (остановка при достижении bakStopTemp) -> FINISHING_WORK (охлаждение).

Ректификация включает более сложную логику: WATER_TEST -> RAZGON -> WAITING -> NASEBYA (стабилизация) -> VALVE_CAL (калибровка клапанов) или SET_PW_AS (ввод мощности и объема AC) -> GOLOVY (отбор голов) -> GOLOVY_OK (подтверждение оператором) -> TELO (отбор тела) -> FINISHING_WORK. Этап GOLOVY поддерживает два метода: Стандартный (один подэтап, 10% от объема AC) и КСС (три подэтапа: Плевков 2%, Стандарт 3%, Добор 15%). Этап TELO реализует два метода управления: Overhist (гистерезис с циклической стабилизацией) и Шпора (плавное снижение скорости при превышении delta).

Расчет таймингов клапанов выполняется через метод calcValveTiming(), который вычисляет время открытия и закрытия на основе целевой скорости отбора, пропускной способности клапана и минимального времени открытия. Реализована оптимизация износа клапанов: при высокой целевой скорости увеличивается время открытия, что уменьшает количество срабатываний. Давление атмосферы корректирует уставки температуры через коэффициент pressureCoeff (0.028 град/гПа), что компенсирует влияние атмосферного давления на точки кипения.

2.3. AppNetwork - Сетевой менеджер и Web-интерфейс

AppNetwork (AppNetwork.h/cpp, ~1465 строк) отвечает за всю сетевую функциональность: WiFi-подключение, веб-сервер, captive portal, DNS-сервер и интеграцию с облаком. Модуль работает на Core 0 через отдельную FreeRTOS задачу, запускаемую через startTask(). Веб-сервер предоставляет REST API: /api/status (GET - текущее состояние системы в JSON), /api/command (POST - команды управления), /api/settings (POST - сохранение настроек), /api/calcvale (POST - расчет клапанов), /api/saveprofile и /api/loadprofile (работа с профилями), /api/logs (GET - чтение логов с SD).

Особенностью архитектуры является двойной режим WiFi (AP+STA): точка доступа с адресом 192.168.4.1 запускается немедленно в setup() через startWebServerEarly(), а подключение к роутеру происходит параллельно. Если STA-подключение происходит в фоне после неудачной попытки в begin(), система автоматически переключается из AP-режима в STA-режим через флаг switchedToSTA, что позволяет инициализировать CloudManager уже после успешного WiFi-соединения. Для защиты от блокировки

основного цикла, все HTTP-обработчики защищены флагом `systemReady`, который устанавливается только после полной инициализации системы.

JSON в API формируется вручную через конкатенацию строк (`String += ...`), что позволяет избежать зависимости от тяжелых JSON-парсеров, но создает фрагментацию кучи при частых вызовах. Обработчик `/api/status` формирует один из самых крупных JSON-ответов, включающий более 60 полей с текущим состоянием системы, датчиков, клапанов, телеметрии и калибровки.

2.4. OutputManager - Управление выходами

`OutputManager` (`OutputManager.h/cpp`, ~404 строки) управляет всеми физическими выходами системы: нагревателями (два канала с отдельным контактором), клапанами голов, тела и воды, мешалкой и зуммером. Каждый выход представлен структурой `OutputChannel` с параметрами: пин, тип (НЗ/НО), активный уровень и текущее состояние. Метод `write()` автоматически учитывает тип клапана (инвертирует сигнал для нормально-открытых клапанов), что обеспечивает унифицированный интерфейс управления.

Модуль поддерживает циклическое управление клапанами (`startHeadValveCycling/startBodyValveCycling`) для импульсного отбора. Цикл реализован через модульную арифметику (оператор `%`) над разницей между текущим временем и временем старта, что не требует хранения состояния каждого цикла. Также реализована циклическая работа мешалки (`startMixerCycling`) и звуковая сигнализация через конечный автомат `BuzzerSM` с тремя фазами (ON, пауза между бипами, пауза перед повтором). Метод `emergencyStop()` немедленно обесточивает все реле и включает постоянный сигнал зуммера, а `safeShutdown()` реализует плавное завершение с охлаждением через подачу воды на configurable время.

2.5. SensorManager - Управление датчиками

`SensorManager` (`SensorManager.h/cpp`, ~248 строк) реализует singleton для управления четырьмя датчиками DS18B20. Каждый датчик имеет индивидуальный фильтр скользящего среднего (окно 5 отсчетов), флаг калибровки, адрес на шине 1-Wire и метку времени последнего валидного чтения. Опрос датчиков выполняется асинхронно: `requestTemperatures()` запускает преобразование, а через 800 мс результат считывается и фильтруется.

Модуль поддерживает процедуру калибровки датчиков: при вызове `startCalibration()` выполняется сканирование всей шины 1-Wire, запоминаются начальные температуры всех найденных устройств, затем в течение 30 секунд система ожидает нагрева идентифицируемого датчика (порог +5 градусов). При обнаружении адрес автоматически сохраняется в EEPROM через `ConfigManager`. Реализована проверка на дубликаты: если датчик уже привязан к другому слоту, он не будет предложен для калибровки текущего слота.

2.6. SDLogger - Логирование на SD-карту

`SDLogger` (`SDLogger.h`, ~225 строк) обеспечивает ротируемое логирование на SD-карту с потокобезопасным доступом через RAII-обертку `SDScopeLock`, которая автоматически захватывает и освобождает мьютекс `sdMutex` при входе и выходе из области

видимости. Это гарантирует, что логирование из разных FreeRTOS задач (Core 0 и Core 1) не приведет к повреждению файловой системы.

Размер лог-файла ограничен 100 КБ, при превышении которого выполняется ротация: system.log переименовывается в system1.log, предыдущие логи сдвигаются (1->2, 2->3 и т.д.), system5.log удаляется. Для оптимизации производительности используется отслеживаемый размер currentFileSize вместо вызова file.size() на каждую запись, что значительно снижает нагрузку на SD-карту. Для чтения логов через веб-интерфейс используется readLastLog() с буфером 16 КБ и readNewLog() с инкрементным чтением для облака (до 2 КБ за раз).

2.7. CloudManager - Облачное подключение

CloudManager (CloudManager.h, ~505 строк) реализует асинхронное облачное подключение через отдельную FreeRTOS задачу на Core 0 со стеком 16 КБ (требуется для SSL/TLS). Ключевой архитектурный принцип: update() в основном цикле (Core 1) НЕ выполняет HTTP-запросы. Он только копирует телеметрию в общий буфер через мьютекс и читает очереди команд/настроек. Все HTTP-запросы выполняются в отдельной cloud задаче. При падении облачного сервера блокируется ТОЛЬКО cloud задача, основной цикл работает без задержек.

Реализован экспоненциальный backoff при ошибках: множитель интервала удваивается при каждой ошибке (максимум до 2 в степени 5 = 32x), что позволяет системе пережить длительные сбои сервера без перегрузки сети. HTTPS-клиент инициализируется с setInsecure() (без проверки сертификата), что упрощает deployment но создает уязвимость для MITM-атак. Защита от переполнения очередей: при заполнении очереди команд или настроек выводится предупреждение в Serial. Мониторинг свободного стека выполняется каждые 60 секунд, при падении ниже 2048 байт выводится предупреждение.

2.8. ConfigManager - Хранение настроек

ConfigManager (preferences.h/cpp) хранит все настройки в EEPROM объемом 512 байт, используя отображение адресов через enum EEPROMAddress. Структура SystemConfig содержит более 50 параметров, включая температурные уставки, тайминги клапанов, пропускные способности, адреса датчиков (по 8 байт каждый), параметры облака и инженерные настройки (скорости отбора, доли голов, коэффициенты). Для изменения настроек через веб-интерфейс используется ручной парсинг JSON через substring() и indexOf() - каждый параметр извлекается индивидуально с дефолтным значением из текущей конфигурации, что позволяет частичное обновление настроек.

3. Анализ процессов дистилляции и ректификации

3.1. Дистилляция: этапы и логика

Процесс дистилляции реализован как последовательный конечный автомат с семью основными этапами. WATER_TEST проверяет наличие воды в дефлегматоре: открывается клапан подачи воды и, если включена опция, запускается мешалка.

Оператор подтверждает наличие воды кнопкой SET (DIALOG_YES), после чего начинается RAZGON - нагрев куба. При достижении температуры razgonTemp (настраивается, по умолчанию 78 градусов) нагрев переключается с полной мощности на рабочую, открывается подача воды, и система переходит в WAITING.

WAITING ожидает прогрева царги на 5 градусов от начальной температуры TSAR, что является критически важным этапом для стабилизации колонны перед началом отбора. В DIST-режиме при достижении этого условия система переходит к OTBOR. На этапе OTBOR контролируется температура бака: при достижении midterm (температура смены посуды) с коррекцией по атмосферному давлению (760 мм рт.ст. как база, коэффициент 0.037 град/мм рт.ст.) клапан закрывается и система переходит в REPLACEMENT для смены тары. При достижении bakStopTemp (обычно 98-99 градусов) система переходит в BAKSTOP, где питание ТЭНов отключается, но контактор остается включенным для поддержания циркуляции воды.

Реализована интеллектуальная обработка потери/восстановления датчика BME280: при старте OTBOR захватывается референсное давление, и если BME пропадает во время процесса, давление замораживается на последнем известном значении, а при восстановлении - обновляется. Это предотвращает резкие изменения уставок midterm из-за скачков давления при перезагрузке датчика.

3.2. Ректификация: этапы и логика

Ректификация реализует значительно более сложный конечный автомат по сравнению с дистилляцией. После WATER_TEST и RAZGON система переходит в NASEBYA (насыщение/стабилизация колонны), длительность которого настраивается (по умолчанию 30 минут). При первом входе в NASEBYA (counter=0) система может направить на калибровку клапанов (VALVE_CAL) или ввод параметров мощности и объема (SET_PW_AS), в зависимости от флага calibration в настройках.

Этап GOLOVY (отбор голов) является ключевым и поддерживает два метода. Стандартный метод рассчитывает объем голов как 10% от объема абсолютного спирта и отбирает их с постоянной скоростью через импульсное управление клапаном. Метод КСС (Комплексная Стабилизация Спирта) реализует трехподэтапный отбор: Плевков (2% от АС, клапан открыт постоянно), Стандарт (3%, импульсный режим), Добор/Акатело (15%, скорость тела). Для каждого подэтапа рассчитывается целевой объем и время, а накопленный объем голов отслеживается в реальном времени через интегрирование скорости по времени (метод Эйлера).

Этап TELO (отбор тела) реализует два метода управления. Метод Overhist использует гистерезис: при превышении температуры TSAR на величину histeresis от референсного значения rtsarM (зафиксированного при старте TELO) клапан закрывается, счетчик циклов увеличивается, и при достижении cycleLim система переходит к повторной стабилизации (NASEBYA с таймером reklamTime). Метод Шпора (доступен только для NC-клапана при cycleLim=1) реализует плавное снижение скорости отбора при превышении delta-порога с коэффициентом shporaCorr (по умолчанию 1.13) и паузой стабилизации shporaStabMs (60 секунд). Это позволяет более точно отслеживать момент залета и минимизировать потери качества продукта.

3.3. Система безопасности (Safety)

Система безопасности реализована в методе `ProcessEngine::checkSafety()` и контролирует два параметра. Мониторинг TSA (температура в царге): при превышении `tsaLimit` запускается таймер обратного отсчета (`emergencyTime` минут), в течение которого звучит прерывистый сигнал зуммера (3 коротких сигнала каждые 8 секунд). Если температура не снижается за отведенное время, система переходит в `FINISHING_WORK` (аварийное завершение). Мониторинг BOX (температура корпуса): при превышении `boxMaxTemp` включается предупреждение (2 коротких сигнала каждые 30 секунд). При недоступности датчика TSA проверка тревоги не срабатывает, что предотвращает ложные срабатывания при обрыве провода датчика, однако это также означает, что отказ датчика TSA остается незамеченным до ручной проверки.

4. Выявленные баги и уязвимости

4.1. Утечки памяти и фрагментация кучи

[HIGH] Строковая конкатенация в `buildTelemetryJson()` и `handleApiStatus()`

В `AppNetwork.cpp` JSON формируется через последовательную конкатенацию `String` объектов (`json += ...`). Вызов `buildTelemetryJson()` происходит каждые 2 секунды при активном облачном подключении и создает временные строки общим объемом 2-4 КБ за вызов. `Arduino String` использует динамическое выделение памяти (`malloc/realloc`), и при частом создании/уничтожении больших строк возникает фрагментация кучи ESP32. На платформе с 512 КБ SRAM это может привести к нехватке памяти для больших буферов (например, при формировании ответа `/api/logs` с 16-КБ буфером). Решение: использовать предвыделенный `char` buffer с `snprintf()` вместо конкатенации `String`, или JSON-библиотеку с фиксированным буфером (`ArduinoJson`).

[HIGH] Статические переменные в `handleTelo()`

В методе `ProcessEngine::handleTelo()` (строка 1338-1343) объявлены статические переменные: `bmeWasAvailable`, `bmeReferenceCaptured`, `lastOpenMs`, `lastCloseMs`. Эти переменные сохраняют состояние между вызовами, что является правильным паттерном для `first-entry` логики, однако статические переменные в методе класса - это антипаттерн в C++. Они создаются один раз и разделяются между всеми экземплярами класса (хотя в данном случае класс `singleton`). Проблема в том, что после завершения процесса и начала нового, эти переменные не сбрасываются, что может привести к использованию устаревших данных. Например, `bmeWasAvailable` может остаться `false` после потери BME280 в предыдущем процессе, что повлияет на логику захвата референса давления.

[MEDIUM] `logger.log()` создает временные `String` при каждом вызове

Каждый вызов `logger.log(message)` создает несколько временных `String` объектов: `timeStr`, `logLine`, и строку формата. В системе `logger.log()` вызывается очень часто (каждое событие безопасности, каждое изменение этапа, периодические обновления). На ESP32 каждый `malloc/free` фрагментирует кучу. Рекомендация: использовать предварительно выделенный кольцевой буфер (`ring buffer`) для логов и записывать на SD блоками, минимизируя выделения в куче.

4.2. Риск зависания системы

[CRITICAL] Блокирующие `delay()` в `connectToWiFi()`

В `AppNetwork.cpp::connectToWiFi()` (строки 233-274) используются вложенные циклы с `delay(100)` для ожидания подключения WiFi. При 3 попытках по 2 сети с 15 итерациями по 100 мс, общее время блокировки может составить до 18 секунд. Хотя внутри циклов вызывается `server->handleClient()` и `yield()`, на Core 0 это может привести к пропуску HTTP-запросов от клиента (веб-интерфейс перестанет отвечать). В сочетании с периодической проверкой WiFi (каждые 30 секунд в STA-режиме), это может вызывать регулярные зависания веб-интерфейса на время переподключения.

[HIGH] Блокирующий `beep()` в `OutputManager`

Метод `OutputManager::beep()` (строка 152-159) использует `delay()` внутри цикла для генерации звуковых сигналов. При `count=3` и `durationMs=100` это блокирует поток на 500 мс. Хотя `beep()` используется только для ручного управления (не из автоматических процессов), если он будет случайно вызван из `processEngine.update()` на Core 1, это приведет к пропуску циклов управления, что особенно опасно на быстротечных этапах (GOLOVY, TELO), где пропуск цикла может привести к переливу или недоливу продукта. Рекомендация: реализовать неблокирующую генерацию сигналов через конечный автомат (аналогично `BuzzerSM`, который уже реализован для тревог).

[MEDIUM] `delay(1000)` в `printStartupInfo()`

В методе `ProcessEngine::printStartupInfo()` (строки 658-661) используются два вызова `sensorAdapter->update()` с `delay(1000)` между ними для прогрева датчиков. Это блокирует выполнение `setup()` на 2+ секунды. Хотя это происходит только один раз при старте, суммарно с другими `delay()` в `setup()` время инициализации может быть значительным (5-10 секунд), в течение которых система не реагирует на внешние события.

4.3. Проблемы многозадачности и синхронизации

[HIGH] Гонка данных (race condition) при чтении настроек

`ConfigManager` использует одну копию `SystemConfig`, к которой обращаются оба ядра: Core 1 через `processEngine` (чтение параметров в реальном времени) и Core 0 через `AppNetwork` (запись настроек из веб-интерфейса через `handleApiSettings`). Хотя запись в EEPROM атомарна на уровне отдельных полей, чтение и запись составных параметров (например, `midterm + midterm_abv + pressureCoeff` одновременно) не атомарны. Если в процессе записи Core 0 прервется на середине, Core 1 может прочитать несогласованное состояние (новый `midterm` со старым `midterm_abv`). Решение: использовать двойную буферизацию (double buffering) или мьютекс для доступа к конфигурации.

[MEDIUM] `xQueueSend` с нулевым таймаутом в `CloudManager` callback

В `BuhloWar.ino` (строки 219-242) при получении команды от облака через callback `CloudManager::onCommandReceived`, команда отправляется в `commandQueue` через `xQueueSend(commandQueue, &msg, 0)` с нулевым таймаутом. Если очередь полна (32 элемента), команда молча отбрасывается. В нормальной работе очередь не переполняется, но при быстрой серии команд от облака (например, при восстановлении связи после длительного сбоя, когда накопились команды) критически важные команды (STOP, START) могут быть потеряны.

[MEDIUM] Дублирование lambda-обработчиков для `CloudManager`

В BuhloWar.ino код регистрации cloud callback дублируется дважды: один раз в setup() (строки 216-243) для начального подключения к WiFi, и второй раз в loop() (строки 322-349) для фонового перехода AP->STA. Эти два блока кода идентичны. Если логика callback изменится в одном месте, ее нужно не забыть обновить и в другом. Рекомендация: вынести регистрацию callback в отдельную функцию registerCloudCallbacks().

4.4. Уязвимости безопасности

[HIGH] HTTP API без аутентификации

Все API endpoints (/api/command, /api/settings, /api/calvalve и др.) доступны без какой-либо аутентификации. Любое устройство в локальной сети может отправить POST-запрос на /api/command с командой STOP_PROCESS или START_RECT, что приведет к остановке или запуску процесса. В AP-режиме (192.168.4.1) это означает, что любой, кто подключится к точке доступа ESP32, может управлять нагревателями и клапанами 220В. Рекомендация: реализовать базовую HTTP-аутентификацию (Basic Auth или токен) для всех mutating endpoints.

[HIGH] Слабый пароль AP по умолчанию

В config.h (строка 63) жестко задан пароль точки доступа: AP_PASS = '12345678'. Этот пароль используется для подключения к ESP32 в AP-режиме и не может быть изменен пользователем через веб-интерфейс (нет соответствующего API endpoint). Любой, знающий этот пароль по умолчанию (он открыт в исходном коде на GitHub), может подключиться к устройству и управлять им. Рекомендация: генерировать уникальный пароль при первом запуске и сохранять в EEPROM, добавить API для его изменения.

[MEDIUM] _client.setInsecure() в CloudManager

В CloudManager.h (строка 68) HTTPS-клиент инициализируется с _client.setInsecure(), что отключает проверку SSL-сертификата облачного сервера. Это делает систему уязвимой для атак man-in-the-middle: attacker в той же сети может перехватить трафик между ESP32 и облачным сервером, подменив ответы (например, отправив ложную команду STOP или START). Рекомендация: хотя полная проверка сертификатов на ESP32 затруднена из-за проблем с актуальностью корневых сертификатов, следует хотя бы реализовать пиннинг (certificate pinning) для конкретного сервера.

4.5. Логические ошибки управления

[HIGH] Отключение Safety при отсутствии датчика TSA

В ProcessEngine::checkSafety() (строка 167) условие data.isTsaValid() предотвращает срабатывание тревоги при невалидном датчике TSA. Это означает, что при обрыве провода датчика TSA (самый критически важный датчик безопасности!) система не обнаружит перегрев царги и не перейдет в аварийный режим. Хотя это предотвращает ложные срабатывания, для системы, управляющей нагревателями 220В с легковоспламеняющимися парами, это создает серьезную угрозу безопасности. Рекомендация: при потере валидности датчика TSA немедленно инициировать аварийное завершение (FINISHING_WORK), а не игнорировать проверку.

[MEDIUM] closeMs < 0 в calcValveTiming при low duty cycle

В ProcessEngine::calcValveTiming() (строки 981-1019) при маленьком duty cycle и большом minOpen может возникнуть ситуация, когда idealOpenMs < minOpen, но

closeMsCalc вычисляется корректно. Однако при экстремально малых целевых скоростях (близких к нулю) closeMs может стать очень большим (сотни тысяч миллисекунд), что приведет к практически постоянному закрытому состоянию клапана. Хотя минимальная граница closeMs < 100 установлена, целевая скорость 0 мл/ч может привести к делению на ноль или близкое к нулю значение в формуле $dutyCycle = targetSpeed / (capacity * 60)$. Рекомендация: добавить явную проверку `targetSpeed > 0` перед вычислениями.

[MEDIUM] Потеря текущего времени при переполнении millis()

ESP32 использует 32-битный микросекундный таймер, который переполняется примерно через 49.7 дней (4294967295 мкс). Все вычисления, основанные на millis() - stageTimeSec, processTimeSec, циклы клапанов, таймеры тестов - будут содержать ошибку при переполнении. Для длительного процесса ректификации (занимающего несколько дней непрерывно) это может привести к неправильному определению времени этапа или объема отбора. Хотя на практике процесс редко длится более 24 часов, это потенциальная проблема. Решение: использовать xTaskGetTickCount() или отслеживать переполнение.

5. Сводная таблица проблем

#	Проблема	Критичность	Модуль	Влияние
1	Строковая конкатенация JSON (фрагментация кучи)	HIGH	AppNetwork.cpp	Утечка памяти, нестабильность
2	Статические переменные в handleTelo() без сброса	HIGH	ProcessEngine.cpp	Устаревшие данные при новом процессе
3	Блокирующие delay() в connectToWiFi()	CRITICAL	AppNetwork.cpp	Зависание веб-интерфейса до 18 сек
4	Блокирующий beep() в OutputManager	HIGH	OutputManager.cpp	Пропуск циклов управления
5	Race condition при чтении/записи конфигурации	HIGH	AppNetwork + ProcessEngine	Несогласованное состояние
6	HTTP API без аутентификации	HIGH	AppNetwork.cpp	Удаленное управление без пароля
7	Слабый пароль AP (12345678)	HIGH	config.h	Открытый доступ к устройству
8	Safety отключен при отсутствии датчика TSA	HIGH	ProcessEngine.cpp	Отсутствие защиты от перегрева
9	_client.setInsecure() в CloudManager	MEDIUM	CloudManager.h	MITM-уязвимость
10	xQueueSend с нулевым таймаутом	MEDIUM	BuhloWar.ino	Потеря команд при переполнении

#	Проблема	Критичность	Модуль	Влияние
1 1	Дублирование cloud callback кода	MEDIUM	BuhloWar.ino	Риск рассинхронизации
1 2	logger.log() создает временные String	MEDIUM	SDLogger.h	Фрагментация кучи
1 3	delay(1000) в printStartupInfo()	MEDIUM	ProcessEngine.cpp	Задержка инициализации
1 4	closeMs < 0 при экстремально low duty cycle	MEDIUM	ProcessEngine.cpp	Некорректные тайминги клапанов
1 5	Переполнение millis() через 49.7 дней	LOW	Все модули	Ошибка времени при длительном процессе

Таблица 2. Полный список выявленных проблем с оценкой критичности

6. Рекомендации по улучшению

6.1. Критические (безопасность)

- **Включить Safety при потере датчика TSA:** При потере валидности датчика TSA (isTsaValid() == false) немедленно инициировать аварийное завершение. Это единственный датчик, который контролирует температуру в царге - его отказ критичен для безопасности.
- **Добавить HTTP-аутентификацию:** Реализовать Basic Auth или Bearer token для всех mutating API endpoints (/api/command, /api/settings). Для чтения (/api/status) аутентификация может быть опциональной.
- **Генерировать уникальный пароль AP:** При первом запуске генерировать случайный пароль из 8 символов, сохранять в EEPROM, добавить API endpoint для его изменения.

6.2. Высокий приоритет (стабильность)

- **Заменить String-конкатенацию на fixed buffer:** В buildTelemetryJson() и handleApiStatus() использовать предварительно выделенный char-буфер с snprintf(). Это устранил фрагментацию кучи и ускорит формирование JSON.
- **Сбрасывать статические переменные handleTelo():** При вызове changeStage() из TELO явно сбрасывать bmeWasAvailable, bmeReferenceCaptured, lastOpenMs, lastCloseMs.
- **Асинхронное подключение WiFi:** Заменить блокирующие delay() в connectToWiFi() на неблокирующее ожидание с использованием WiFi.eventHandled() или FreeRTOS таймеров.
- **Неблокирующий beep():** Заменить delay()-based beep() на BuzzerSM-подобный конечный автомат для всех звуковых сигналов.
- **Мьютекс для ConfigManager:** Добавить мьютекс для доступа к SystemConfig, особенно для операции чтение-модификация-запись.

6.3. Средний приоритет (улучшения)

- **Обработка переполнения millis():** Отслеживать переполнение таймера и корректировать все вычисления на основе millis() или использовать xTaskGetTickCount().
- **Вынести cloud callback в функцию:** Избавиться от дублирования кода регистрации onCommandReceived callback.
- **xQueueSend с таймаутом:** Использовать таймаут 100-500 мс вместо 0, чтобы при полной очереди хотя бы пытаться дождаться освобождения.
- **Оптимизация SDLogger:** Использовать кольцевой буфер для накопления логов и запись блоками на SD, снижая количество операций ввода-вывода и выделений памяти.
- **SSL certificate pinning:** Для CloudManager реализовать проверку отпечатка сертификата облачного сервера вместо setInsecure().
- **Проверка targetSpeed > 0:** В calcValveTiming() добавить явную проверку целевой скорости перед делением.